Verantwoordingsdocument

Restaurantreserveringen, bestellingen en menukaart

Ali Khoshkam

Leerlijn Backend (30 EC) – Eindopdracht versie 4.3

Inleverdatum: 8 november 2026

Documentversie: 1.1 | LaTeX-bron 8 november 2026

Inhoudsopgave

1 Inleiding en status **2**

2 Implementatiekeuzes **2**
2.1 Tafel vergrendelen vóór overlapcontrole . . . 2
2.2 Prijzen en namen als momentopname . . . 2
2.3 Identity buiten het domein . . . 2
2.4 DTO-mapping binnen de transactie . . . 3
2.5 Afbeeldingsbytes in de database . . . 3
2.6 Factuur als grens voor wijzigingen . . . 3

3 Limitaties en vervolg **3**
3.1 Vaste reserveringsduur en één tafel . . . 3
3.2 Geen keukenworkflow of voorraad . . . 4
3.3 Eenvoudige afrekening . . . 4
3.4 Geen bevestigingen of herinneringen . . . 4
3.5 Beperkte zoek- en beheermogelijkheden . . . 4
3.6 Lokale infrastructuur en media . . . 4

4 Bijlage — AI, bronnen en inlevering **4**
4.1 Gebruikte gebruikersprompts . . . 5
4.2 Bronnen . . . 5

1 Inleiding en status

In dit document leg ik uit welke keuzes ik in de restaurant-API heb gemaakt, en wat ik bewust nog niet heb gebouwd. Het zijn zes technische keuzes en zes beperkingen. Achterin staan de bronnen en de prompts die ik aan de AI heb gegeven.

Persoonlijke reflectie. Ik heb gekozen voor een restaurantapplicatie omdat ik altijd al het idee heb gehad dat het leuk zou zijn om ooit mijn eigen restaurant te hebben (misschien gebeurt dat ooit nog in de toekomst). Daarom sprak dit onderwerp mij persoonlijk aan. Tijdens het ontwikkelen heb ik gebruikgemaakt van AI om mij te helpen met programmeren. Omdat veel onderdelen nog nieuw voor mij zijn, vond ik het soms lastig om verder te komen. Ik heb een aantal keer opnieuw moeten beginnen omdat ik ergens niet uitkwam. Ik ben vooral trots op hoe overzichtelijk de app is. Ik vind het belangrijk dat een gebruiker gemakkelijk kan begrijpen hoe de applicatie werkt en geen tijd verliest met het zoeken naar het juiste scherm. Als ik de applicatie verder zou ontwikkelen, zou ik online betalingen toevoegen. Wanneer ik zelf uit eten ga, vind ik het soms vervelend dat ik lang moet wachten om af te rekenen omdat het personeel druk is. Met een online betaalmogelijkheid kunnen klanten zelf betalen wanneer ze willen.

De code staat publiek op `https://github.com/Alikhoshkam1989/myRestaurant`. Hoe je de API start, staat in `README.md`. De testresultaten staan in `docs/verification.md`. Vijf pull requests die echt naar `main` zijn gemerged, heb ik nog niet. Lokale commits tellen daar niet voor.

2 Implementatiekeuzes

2.1 Tafel vergrendelen vóór overlapcontrole

Twee verzoeken kunnen allebei zien dat een tafel vrij is en daarna allebei opslaan. Alleen een SELECT is dan niet genoeg. In `ReservationService.apply()` haal ik de tafel daarom eerst op met `findLockedById()`. Die lock geldt voor die ene tafel. Een andere tafel kan intussen gewoon geboekt worden.

Technische reflectie. Een `synchronized` in Java beschermt alleen dit ene programma. De lock in de database blijft gelden als de API later met meer dan één proces draait. In de test met twee boekingen tegelijk slaagt er precies één. Hoe lang een verzoek op die lock wacht, heb ik niet gemeten.

Ik vind het belangrijk dat twee klanten niet per ongeluk dezelfde tafel kunnen reserveren. Dit zou voor vervelende situaties zorgen, zowel voor de klanten als voor het restaurant.

2.2 Prijzen en namen als momentopname

Op `OrderLine` sla ik `unitPrice` en `productName` op naast het product. `OrderService` kopieert die velden op het moment dat de regel wordt toegevoegd. Wijzig ik de bestelling later, dan blijven bestaande regels op die oude prijs en naam. Het totaal reken ik uit die regels, met `BigDecimal`. Met `double` vertrouwde ik de centen niet.

Technische reflectie. Als ik bij het afrekenen steeds de prijs van dit moment uit de menukaart zou lezen, schuift een oude bestelling mee zodra iemand de kaart aanpast. De extra kolommen kosten opslag. Daarvoor krijg ik een rekening die ik nog kan uitleggen. Een nieuwe prijs geldt pas voor een nieuwe regel.

Ik vind het belangrijk dat klanten altijd het bedrag betalen dat op de menukaart stond toen ze bestelden, en niet dat de rekening plots verandert omdat het restaurant prijzen aanpast.

2.3 Identity buiten het domein

Wachtwoorden wilde ik niet in mijn eigen tabel. Keycloak doet de registratie, het wachtwoord en de rollen. In `Reservation` bewaar ik alleen `customerSubject`, uit het JWT dat al gecontroleerd

is. Een klant kan in de body geen rol meesturen en ook niet zeggen dat een reservering van iemand anders is. Het token moet audience `restaurant-api` hebben.

Technische reflectie. Een eigen wachtwoordtabel betekent dat ik hashing en resets zelf moet bijhouden, naast de rest van de opdracht. Dat heb ik niet gedaan. Het subject blijft hetzelfde als iemand zijn naam wijzigt. Wat ik nog niet heb: als een account verdwijnt, wat er dan met oude reserveringen gebeurt.

Mensen die een account aanmaken moeten erop kunnen vertrouwen dat hun persoonlijke gegevens beschermd zijn. Keycloak is hier een betrouwbare oplossing voor.

2.4 DTO-mapping binnen de transactie

`Views.of()` roep ik in de service aan, terwijl de transactie nog open is. De controller krijgt alleen records terug. `open-in-view` staat uit, dus een entiteit gaat niet mee naar buiten.

Technische reflectie. Eerst leek mappen in de controller overzichtelijker. Dat ging mis zodra Hibernate na de transactie nog een categorie of een regel wilde laden. Door het in de service te doen, is de response al een gewoon object voordat de transactie dichtgaat. Voor de menukaart van dit project is dat genoeg. Bij een heel lange lijst zou ik de query zelf alleen de velden laten ophalen die de response nodig heeft.

2.5 Afbeeldingsbytes in de database

Een product heeft hooguit één foto, in `ProductImage`. Ik kijk naar de bytes zelf: is het PNG of JPEG, gaat het plaatje open, en blijft het onder 2 MiB en 16 miljoen pixels. De bestandsnaam uit de upload gebruik ik nergens als pad. Naam en inhoud schrijf ik in dezelfde transactie weg.

Technische reflectie. Een map op schijf leek eerst simpeler. Dan kunnen de database en de map uit elkaar lopen, en een naam als `../` wil ik niet als pad gebruiken. Voor de foto's op deze menukaart is de database prima. Een grote verzameling foto's zou ik later buiten de database zetten.

2.6 Factuur als grens voor wijzigingen

`InvoiceService` lockt de bestelling en maakt maximaal één factuur. Daarna weigert `OrderService` elke wijziging. `paid` zet ik niet terug op onbetaald. Een factuur die nog niet betaald is, mag wel weg. Dan kan de bestelling weer aangepast worden.

Technische reflectie. Zonder die grens kan iemand de bestelling veranderen terwijl de factuur het oude bedrag nog toont. Deelbetaling en een creditnota heb ik niet gebouwd. Dat werd te groot naast de rest. Online betalen wil ik later wel, zoals ik in de inleiding schreef. Eerst moet het bedrag op de factuur blijven kloppen met de bestelling.

3 Limitaties en vervolg

3.1 Vaste reserveringsduur en één tafel

Elke reservering duurt bij mij twee uur en gebruikt één tafel. Een gezelschap dat twee tafels nodig heeft, of iemand die drie uur blijft, kan ik zo niet plannen.

Vervolg. Duur per reservering, openingstijden en tafels die je combineert. De overlapcontrole moet dan opnieuw, want die rekent nu met een vast blok van twee uur.

Reflectie. Ik heb de duur vastgezet omdat ik anders niet zeker wist of twee boekingen elkaar raken. Eerst wilde ik de controle kunnen testen. De echte planningsregels zou ik pas vastleggen na een gesprek met het restaurant.

3.2 Geen keukenworkflow of voorraad

Een bestelling is bij mij een lijst regels plus een totaal. Er is geen status zoals “in de keuken” of “op tafel”, en ik tel geen ingrediënten.

Vervolg. Statussen op de bestelling, een keukenoverzicht en voorraad die meebeweegt als er iets besteld wordt.

Reflectie. `available` op een product is alleen aan of uit. Als de tiramisu op is tijdens een drukke avond, ziet de API dat niet. Ik heb voorraad gelaten omdat bestellingen, facturen en foto’s er al in moesten, en ik onderweg een paar keer opnieuw ben begonnen.

3.3 Eenvoudige afrekening

De factuur heeft een bedrag, een datum en `paid`. Geen btw-regels, geen deelbetaling en geen betaalprovider.

Vervolg. Eerst factuurregels met het tarief van dat moment, daarna pas een betaalprovider. Anders hangt de betaling aan een bedrag dat ik niet kan uitsplitsen.

Reflectie. Een boolean was het kleinste dat ik nodig had om een betaalde rekening op slot te zetten. Voor een echt restaurant is dat te weinig. Online betalen noemde ik al; dat komt pas nadat het bedrag op de factuur blijft kloppen.

3.4 Geen bevestigingen of herinneringen

De reservering staat alleen in de API. Er gaat geen mail of sms uit, en een wachtlijst is er niet.

Vervolg. Een wachtrij die het bericht opnieuw probeert, en een wachtlijst waarin ik zelf een tafel toewijs. Niet automatisch, want dan kan dezelfde tafel weer dubbel weg zijn.

Reflectie. Ik wil geen mail sturen voor een boeking die daarna wordt teruggedraaid. Het bericht mag pas weg als de transactie gelukt is. Voor deze versie was het genoeg dat de klant de reservering in de app terugziet.

3.5 Beperkte zoek- en beheermogelijkheden

Lijsten geven alles terug. Er is geen paginering, geen filter op datum en geen manier om een account te verwijderen.

Vervolg. Paginering, een datumfilter en een afspraak over wat er met oude reserveringen gebeurt als iemand zijn account weghaalt.

Reflectie. Met de demodata merk je niet dat een lijst zwaar wordt. Na een jaar reserveringen wel. Eerst de lijst kleiner maken, daarna pas kijken of een query te langzaam is. Dat heb ik nu niet gemeten.

3.6 Lokale infrastructuur en media

Standaard draait het project op H2, met `ddl-auto=update`, en de foto’s staan in de database. Migraties, monitoring en een geteste backup heb ik niet.

Vervolg. Echte migraties, een backup die ik ook een keer terugzet, en de foto’s uit de database. PostgreSQL staat als optie in de configuratie, maar die route heb ik op een server niet gedraaid.

Reflectie. H2 koos ik zodat ik kon starten zonder eerst een databaseserver te installeren. Dat de tests lokaal slagen, zegt niet dat het op een server blijft staan. Dat moet ik apart proberen.

4 Bijlage — AI, bronnen en inlevering

Ik heb deze opdracht zelf gemaakt. Op 15 en 16 september 2026 heb ik OpenAI Codex om hulp gevraagd als ik vastliep bij de opdracht, een stuk code, een test of deze tekst. De keuzes

hierboven zijn van mij.

4.1 Gebruikte gebruikersprompts

```
1. kun je uitleggen wat ik in deze opdracht zelf moet doen?
2. ik begrijp dit stuk niet. kun je het uitleggen, zodat ik
   het zelf kan schrijven?
```

Aparte subagents heb ik niet gebruikt. Dezelfde prompts staan in `docs/ai-prompts.md`.

4.2 Bronnen

- Ali Khoshkam: *idee fase backend 2.pdf*.
- NOVI: *Eindopdracht Backend v4.3.pdf*.
- Spring Boot: https://docs.spring.io/spring-boot/3.5/system-requirements.html
- Spring Security: https://docs.spring.io/spring-security/reference/6.5/servlet/oauth2/resource-server/jwt.html
- Keycloak: https://www.keycloak.org/server/importExport

Inleverdatum: 8 november 2026. De GitHub-link staat in de inleiding. Nog niet rond: vijf gemergede pull requests, en het bewijs dat de ideefase een GO heeft gekregen.